

A Browser with f1r3lang as its Native Execution Mechanism

Architecture proposal and review of browser component frameworks

Version 0.2 (draft for review)

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

28 September 2026

Abstract

This proposal designs a web browser in which JavaScript and its execution environment are replaced by f1r3lang (f1r3—@ng), running on a native RSpace inside the browser client. It first reviews the available browser component frameworks as they stand in September 2026 — Electron, the Chromium Embedded Framework, Tauri and Wry, Servo, Blitz, Ladybird, WebKit and Gecko — and recommends building on **Blitz**, a modular Rust HTML/CSS engine that has no JavaScript engine to remove, with **Servo** held in an isolated process for legacy JavaScript pages. The execution substrate is the existing CampF1R3 executive (K1ndl1ng parser and normaliser, CampF1R3 store, b3110ws interpreter, f1r3x-host), which is already dependency-free, WebAssembly-portable, and deliberately application-agnostic. The architecture then follows from three identifications: a page is a process, the document is a namespace of unforgeable names, and events are data on channels. Security becomes capability discipline rather than same-origin policy, cost accounting becomes a per-page resource budget, deterministic replay becomes a debugger, and graded where-clauses become the programmable arbiter of user-interface races. Version 0.2 adds two dimensions. The first makes a F1R3FLY shard the server: resources are authenticated by content and by consensus state rather than by the channel they arrive on, which removes substitution by a man in the middle, and sessions become shared names bound to browser-held keys, which removes cookies. The second says when to convert between f1r3lang resources and standard Web resources: import pins, export projects, and behaviour never converts. A five-phase development plan, a list of obligations, and the decisions that need an owner close the document.

Contents

1	Summary and recommendation	3
2	Context: what the F1R3FLY codebases already provide	3
2.1	CampF1R3 (F1R3FLY-io/campf1r3)	3
2.2	f1r3node-rust	4
2.3	The fuzzyware directory of publications	4
3	Goals, non-goals and principles	5
3.1	Goals	5
3.2	Non-goals	5
3.3	Principles	5

4	Review of browser component frameworks	5
4.1	Electron	7
4.2	Chromium Embedded Framework	7
4.3	Tauri 2 and Wry	7
4.4	Servo	7
4.5	Blitz	7
4.6	Ladybird	8
4.7	WebKit and Gecko	8
4.8	The reach tier: CampF1R3 in a stock browser	8
5	Recommendation	8
6	Architecture	9
6.1	What is new, and what is not	9
6.2	Process model	10
7	Execution model	10
7.1	A page is a process	10
7.2	Imports are free names	10
7.3	The document is a namespace	11
7.4	Events are data on channels	11
7.5	The frame is the unit of commit	12
7.6	Determinism, replay and the debugger	12
7.7	Cost accounting as a page budget	12
7.8	Graded where-clauses arbitrate the page's races	12
8	The shard as the server	13
8.1	What the Web authenticates, and what it does not	13
8.2	Three kinds of resource	14
8.3	Dynamic responses and the assurance ladder	15
8.4	Sessions without cookies	15
8.5	Three server tiers	16
8.6	Confidentiality	16
8.7	What remains trusted	17
9	Security model	17
10	Compatibility with the JavaScript web	17
11	Integration with existing web infrastructure	18
11.1	Four rules	18
11.2	f1r3lang to the Web: when to project	18
11.3	The Web to f1r3lang: when to pin	19
11.4	Decision table	20
11.5	What the existing infrastructure becomes	20
12	Development plan	20

13 Risks, obligations and open decisions	22
13.1 Risks	22
13.2 Obligations	22
13.3 Open decisions	23

1 Summary and recommendation

The recommended stack, in one line each:

Document engine: Blitz.

Its `blitz-dom` crate combines Stylo (Firefox’s CSS engine), Taffy (layout) and Parley (text), with `html5ever` for parsing, a Vello/wgpu painter, and a `winit/AccessKit` shell [1]. It has no JavaScript engine, so nothing has to be ripped out; the script host is a hole waiting to be filled.

Execution: the CampF1R3 executive.

`kindling-parse`, `kindling-norm`, `campf1r3-core`, `b11l0ws`, `f1r3x` and `f1r3x-host`, unchanged, instantiated once per page. The new code is one crate, the *page bridge*, that grounds a page’s free names to capabilities and translates between channel traffic and DOM mutation.

Legacy web: Servo, out of process.

A quarantined tab type for pages that need JavaScript, with no access to `f1r3lang` capabilities except an explicit message channel. CEF is the fallback if Servo’s compatibility is not sufficient.

Reach: `f1r3x-wasm`.

The same `f1r3lang` pages run in unmodified browsers through the existing `wasm` host and a small loader, so authors are not locked out of an audience while the native browser matures.

Server: a F1R3FLY shard.

Sites are `rho:serve` projects in the node’s versioned registry, keyed by the publisher’s key. Content is addressed by hash, bindings are checked against finalized state, and sessions are names bound to browser-held keys (Section 8).

Integration: pin and project.

Web resources that `f1r3lang` code depends on are pinned by hash; `f1r3lang` resources are projected to the Web, with their hash and a link home, for audiences and machines that cannot run them (Section 11).

Electron and Tauri are rejected as the base: in both, JavaScript is the application runtime, so `f1r3lang` could only ever be a guest. The reasons, and the full comparison, are in Section 4.

2 Context: what the F1R3FLY codebases already provide

The design leans heavily on work that exists. This section records what was found in the three repositories on 28 September 2026.

2.1 CampF1R3 (F1R3FLY-io/campf1r3)

The repository implements a `rho` executive in roughly 12,600 lines of Rust across eleven crates, with no external dependencies. Every crate except the FFI carries `#! [forbid(unsafe_code)]`. The pieces that matter here:

- `campf1r3-core` is the store: data and waiting continuations at channels, with `produce`, `consume` and `install` as single cuts. Its specification requires `wasm32-unknown-unknown` as a first-

class target, requires observable counters to be fixed-width so that 32-bit wasm and 64-bit native builds agree bit for bit, and forbids `tokio`, `rayon`, `heed` and any C library.

- Choice is isolated in an oracle ω with three profiles: `Free`, `Deterministic` (the default; results and the event log are bit-identical across targets and thread counts) and `Replay` (the store accepts only communications matching a loaded log). This is exactly the seam at which graded resolution attaches (Section 7.8).
- Accounting is hooked at the `COMM` and nowhere else, through an `Observer` trait whose `Err` aborts the operation atomically.
- `f1r3x-host` is a driver written as a state machine rather than a thread: `Session::poll(now.ms)` returns frames, and time is injected. A native host calls it from a thread; a page calls it from `requestAnimationFrame`. Pacing is lockstep or free-running with a bounded buffer, as one rule with two bounds.
- `f1r3x-wasm` already runs the executive in a browser tab, without `wasm-bindgen`, with a working visualiser.
- `k1nd1ng-norm` produces a normal form with de Bruijn indices and ordered parallel components, content-hashed with BLAKE2b-256, so name equality is a byte comparison. The deploy path grounds free names by replacing each with an unforgeable derived from its identifier.

The kernel is stratified: K0 is pure rho, K1 adds polyadic communication, joins, persistence, peek, patterns and `new`, and K2 adds where clauses over the rho-pure-eval expression subset, behind a feature flag. The project memo recorded when the repository was split from `Rhobots-in-Space` states the constraint this proposal relies on: `CampF1R3` must stay application-agnostic so that it can serve many applications, including a browser via `WebAssembly`. Nothing in this proposal adds browser code to that repository.

2.2 f1r3node-rust

The node workspace (`node`, `casper`, `rholang`, `rspace++`, `rho-pure-eval` and others) is the shard side. Two facts shape the design. First, `rspace++` depends on `heed` (LMDB), `tokio` and `rayon`, so it is not the client `RSpace`; `CampF1R3` is. Second, the node exposes an HTTP API (`/api/deploy`, `/api/registry/{uri}`, `/api/blocks`, among others) and an event stream at `/ws/events`, which is sufficient for a first shard bridge (Section 8). The feature branches `feature/cost-accounting-transpiler`, `feature/f1r3lang-mettail-only` and `feature/module-syntax` carry the surface language and its compilation to the kernel; the browser consumes their output rather than embedding them.

2.3 The fuzzyware **directory of** publications

The graded where-clause papers change one thing about the calculus: the where clause of a for-comprehension is valued in a commutative semiring V rather than in the Booleans, and is evaluated per candidate match rather than per communication [11]. The companion notes instantiate V as $\mathbb{R}_{\geq 0}$ (probabilistic logic, and the learner’s gradient in the fibre [12]) and $\mathbb{C}$ (amplitudes and the Born rule [13]). The choice-formulae note adds that for idempotent semirings such as Viterbi ($\max, \times$) and tropical ($\min, +$) the clause is attained by a definable selection function, and that weak fairness is a clause combinator $\psi \oplus \varepsilon$. For a browser, this is the language in which a page says how its races are to be arbitrated.

3 Goals, non-goals and principles

3.1 Goals

1. f1r3lang is the only language a page's behaviour is written in. There is no JavaScript engine in the page process.
2. Each page runs against its own RSpace, in the client, with the same semantics as the shard's.
3. HTML and CSS rendering are standards-based and reused, not reinvented.
4. A F1R3FLY shard is the server: sites are published to it, and the client authenticates what it receives against it.
5. Every page execution is reproducible from its event log.
6. Desktop first (macOS, Linux, Windows); the same core on mobile and headsets later.

3.2 Non-goals

1. Running arbitrary JavaScript pages in the native engine. They go to the legacy tab (Section 10).
2. Reproducing the Web API surface (`window`, `fetch`, `localStorage`) under f1r3lang names. The design replaces it with capabilities.
3. Consensus in the client. A tab's RSpace is local; the shard remains the place where agreement happens.
4. Compiling the full surface language in the browser in version 1. Pages ship kernel normal forms (Section 7.1).

3.3 Principles

Principle 1 (One store, many hosts). The executive is the same code in the native browser, in the wasm reach tier, on the Mac server, and behind the Vision Pro client. The browser is a host in the sense of `HOSTING.md`, not a fork.

Principle 2 (Names are the only authority). A page can affect exactly what it holds names for. No ambient global object exists.

Principle 3 (The document is data in the namespace). DOM nodes are reached through names and changed by communication, never by a foreign-function call that bypasses the store.

4 Review of browser component frameworks

The question each framework was asked is narrow: *can its JavaScript execution environment be replaced, and what does that cost?* A framework whose value lies in its JavaScript runtime is disqualified as the base regardless of its other merits. Status is as of 28 September 2026; sources are listed at the end.

Framework	Engine and language	Can JavaScript be replaced?	Rust embedding	Maturity, Sept. 2026	Licence	Role here
Electron	Chromium and Node.js; C++	No. V8 and Node <i>are</i> the application runtime.	None; Node is the host	Production; a new major every 8 weeks, three supported [4]	MIT, Chromium BSD	Rejected
CEF	Chromium; C/C++, Rust bindings via <code>cef</code> crate	No. V8 is wired through Blink's bindings.	Yes (<code>cef</code>)	Production; tracks Chromium [5]	BSD	Fallback for the legacy tab
Tauri 2 / Wry	OS webview (WKWebView, WebView2, WebKitGTK); CEF runtime in progress	No. Page logic is JavaScript; Rust is only the backend.	Yes	Production, v2.10 [7, 8]	MIT/Apache	Rejected as base
Servo	Rust; SpiderMonkey for JavaScript	Only by rewriting the script crate, which holds the DOM and its JS bindings.	Yes (<code>servo</code> crate, LTS branch)	Pre-1.0; monthly releases, 0.1 LTS in April, 0.5 current [2, 3]	MPL-2.0	Legacy tab, isolated
Blitz	Rust: Stylo, Taffy, Parley, html5ever, Vello	Nothing to replace: it has no JavaScript.	Yes, crate by crate	Beta; renders many no-JS sites [1]	Apache/MIT	Document engine
Ladybird	C++ with a Rust migration; own LibJS	Not a design goal	No	Pre-alpha; public alpha targeted for 2026 [9]	BSD-2	Watch
WebKit (WPE, GTK)	C++; JavaScriptCore	No	C API only	Production	LGPL/BSD	Rejected
Gecko / GeckoView	C++ and Rust; SpiderMonkey	No	Android only	Production	MPL-2.0	Rejected
Stock browser + <code>f1r3x-wasm</code>	Host browser; CampF1R3 as <code>wasm</code>	JavaScript stays as a thin loader	n/a	Working today in <code>campf1r3</code>	Apache-2.0	Reach tier

Table 1: Browser component frameworks against the one question that matters here.

4.1 Electron

Electron embeds Chromium and Node.js into one binary so that a single JavaScript codebase can drive a desktop application [4]. That is its entire value proposition, and it is the opposite of this project's. Replacing V8 would mean maintaining a Chromium fork with Blink's bindings layer rewritten, rebased every eight weeks as Electron tracks Chromium majors. The best that could be done without that is to run CampF1R3 as wasm inside Electron's renderer, which is the reach tier with a heavier wrapper. Electron is rejected.

4.2 Chromium Embedded Framework

CEF embeds a complete Chromium and is the industry default for putting a real browser inside an application; Rust bindings are maintained under the Tauri organisation [5]. Its JavaScript engine is equally unremovable. Its merit here is compatibility: if the legacy tab must render the modern JavaScript web at Chromium fidelity, CEF in a separate process is the dependable answer, at a cost reported by one embedder as a runtime of about 170 MB [6].

4.3 Tauri 2 and Wry

Tauri builds applications from a Rust backend and a web frontend rendered by the operating system's webview, and an official CEF runtime is being built in its repository [8]. Tauri is attractive for application shells, but in every configuration the page's behaviour is JavaScript and Rust is reached through IPC. A Tauri application could host CampF1R3 natively in its Rust half and give pages a bridge to it, but then f1r3lang is an RPC target of JavaScript, which inverts the goal. Tauri is rejected as the base. Two of its crates (muda for menus, tao or winit for windows) are used by Blitz's shell anyway.

4.4 Servo

Servo is a Rust engine under Linux Foundation Europe, now published as a crate with a WebView embedding API and a long-term-support branch every six months, supported for nine months [2]. Monthly releases continue, and 0.5 brought faster text rendering and wider site compatibility [3]. Architecturally it is the closest mainstream engine to what is wanted: Rust throughout, parallel styling, a clean embedder boundary. But its script crate — the DOM implementation and the event loop — is generated from WebIDL onto SpiderMonkey's garbage-collected object model. Removing JavaScript means replacing script, which is the largest and most web-compatibility-sensitive part of the engine. Servo is therefore the right engine for the legacy tab, where JavaScript is wanted, and the wrong one to hollow out.

4.5 Blitz

Blitz is a modular HTML/CSS engine from Dioxus Labs, in beta, that renders many no-JavaScript sites and is usable for applications by early adopters [1]. It is assembled from the same parts as Servo's style system (Stylo, html5ever) plus Taffy for box layout, Parley for text, AccessKit for accessibility and an anyrender abstraction over Vello and other painters. Its authors state that it renders HTML and CSS and deliberately does *not* ship a JavaScript engine, local storage, WebSockets and similar features, leaving them to native crates. Interactivity today comes from Dioxus Native, which drives blitz-dom's tree from Rust. That is precisely the seam f1r3lang needs:

replace the Dioxus virtual DOM with the page bridge, and the executive drives the document. The risks are maturity and the size of the team; they are addressed in Section 13.

4.6 Ladybird

Ladybird is an independent engine built from scratch by a non-profit, reporting over two million passing Web Platform Test subtests and a Rust rewrite of its JavaScript front end in February 2026, with a public alpha targeted for 2026 [9]. It is a browser, not a component; it has no embedding story and its JavaScript engine is one of its central investments. It is worth watching as a future legacy-tab engine and as evidence that a new engine can reach real compatibility, but it is not a base.

4.7 WebKit and Gecko

Both are production engines whose JavaScript engines are woven through their DOM implementations, with C and C++ embedding surfaces (WPE, GeckoView on Android). They offer nothing over CEF for the legacy role and nothing at all for the native one.

4.8 The reach tier: CampF1R3 in a stock browser

`f1r3x-wasm` already runs the executive inside an ordinary tab, driven from the page’s animation loop. A small loader plus a DOM bridge written against the host browser’s DOM gives `f1r3lang` pages an audience on every browser today. JavaScript remains present as the host’s glue, so this tier does not meet Goal 1, but it shares every line of the executive with the native browser, which makes it the natural bootstrap and the natural test oracle.

5 Recommendation

Requirement 1 (Framework choice). The native browser **MUST** use Blitz (`blitz-dom`, `blitz-html`, `blitz-paint`, `blitz-net`, `blitz-shell`) as its document engine and the CampF1R3 executive as its only execution mechanism. Legacy JavaScript pages **SHOULD** be rendered by Servo in a separate process, with CEF as the fallback. The reach tier **SHOULD** be delivered from `f1r3x-wasm`.

Four reasons, in order of weight.

1. **Subtraction versus addition.** Every other engine requires removing a JavaScript engine that its DOM was designed around. Blitz requires adding a script host to a DOM designed to be driven from outside. Addition is bounded; subtraction from a browser engine is not.
2. **One language, one toolchain.** Blitz, CampF1R3 and the node are all Rust, with permissive or file-level-copyleft licences (Apache, MIT, MPL-2.0 for Stylo) compatible with F1R3FLY’s Apache-2.0 practice. There is no C++ build, no Chromium checkout and no rebasing treadmill.
3. **Alignment with the store’s discipline.** CampF1R3 is small, dependency-free and portable by requirement; Blitz is modular by design and already builds for `wasm` (its repository carries a `wasm` how-to). Both can be pinned and vendored, and both can run on a headset.
4. **Shared lineage with the legacy engine.** Blitz and Servo share Stylo and `html5ever`, so CSS behaves the same in native and legacy tabs, and fixes upstream benefit both.

Remark 1. Choosing Blitz is choosing to own more of the engine than an Electron application would. That is intended: the point of the project is that the execution environment of the web is the thing being changed, and it cannot be changed from inside a framework whose purpose is to preserve it.

6 Architecture

The browser is three kinds of operating-system process and one remote party (Figure 1). The *browser process* owns windows, the user’s keys, the network and the capability broker. Each *page process* holds one site’s tabs: a Blitz document per tab and one CampF1R3 executive per tab, joined by the page bridge. A *legacy process*, started only when needed, runs Servo for pages that require JavaScript. The *shard* is reached through the browser process, never directly from a page.

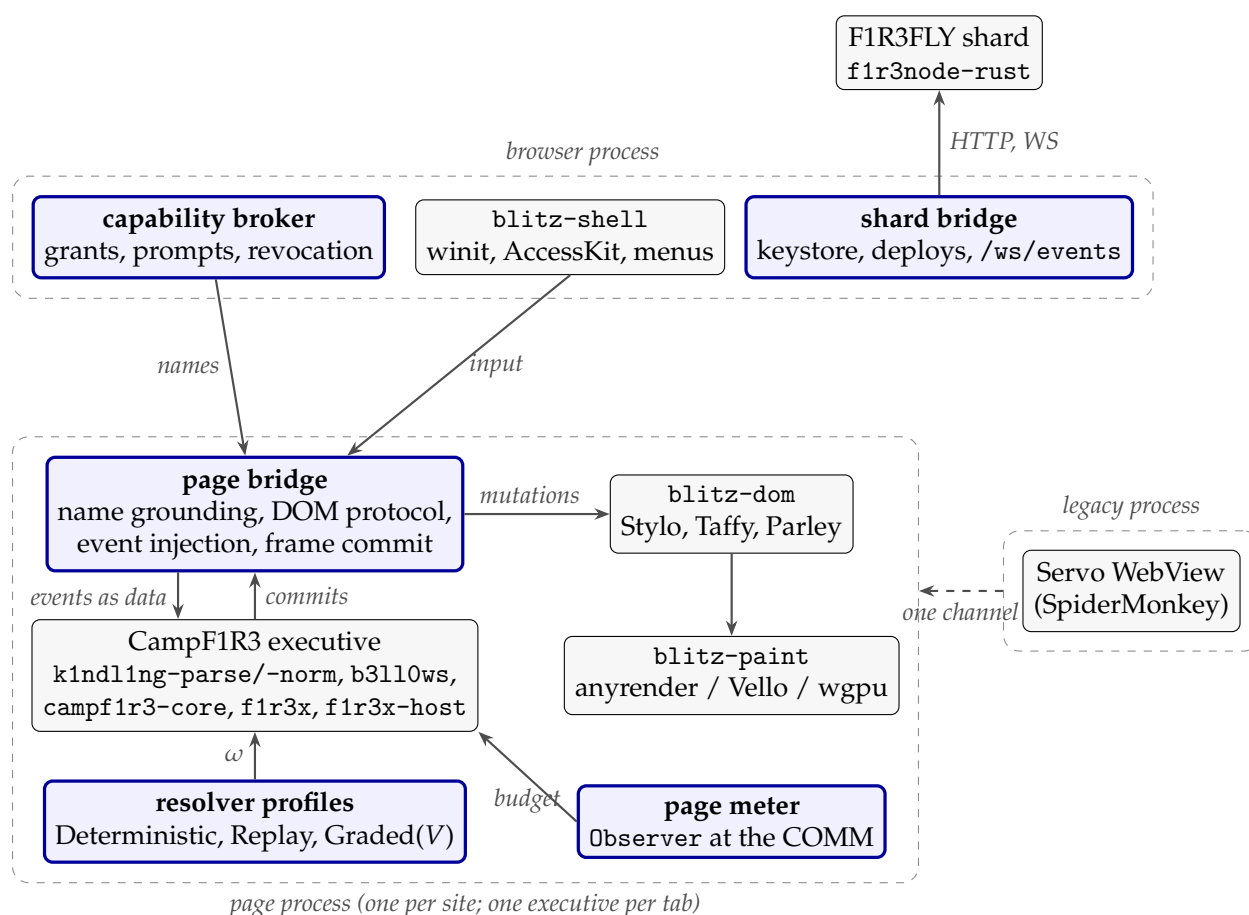


Figure 1: Processes and components. Heavy boxes are new code; light boxes are reused unchanged from campf1r3 and Blitz. The executive never calls the DOM; it communicates, and the bridge turns communications into mutations.

6.1 What is new, and what is not

The new code is four components, all outside the campf1r3 repository:

1. **The page bridge** (§7): grounds free names, speaks the DOM protocol, injects events, and commits a round's mutations to `blitz-dom` once per frame.
2. **The capability broker**: decides which names a page receives, asks the user where a grant needs consent, and can revoke by killing a forwarder.
3. **The shard bridge** (§8): holds keys, signs deploys, proxies registry and data-at-name reads, and turns the node's event stream into produces on local channels.
4. **The resolver and meter profiles** (§7.8, §7.7): a Graded oracle for CampF1R3 and a per-page accounting observer.

Everything else is reused. In particular the executive is linked natively in the page process for speed and multi-core stores, and the *same* crates are what the reach tier ships as wasm.

6.2 Process model

Requirement 2 (Isolation). A page process **MUST** hold pages of one site only. The legacy process **MUST NOT** hold any name granted to a `f1r3lang` page, and **MUST** communicate with page processes only through a single broker-mediated channel carrying ground data.

Rust's memory safety and CampF1R3's `forbid(unsafe_code)` remove the most common reason browsers isolate renderers, but process-per-site is kept because it is also the unit of crash containment, of resource limits enforced by the operating system, and of the legacy engine's quarantine. As defence in depth, the executive **MAY** additionally run as `wasm32` inside the page process under a wasm runtime, since it already builds for that target; whether the cost is worth it is Obligation 2.

7 Execution model

7.1 A page is a process

A `f1r3lang` page is an HTML document whose behaviour is given by processes, carried as

```
<script type="application/f1r3lang" src="counter.knf"
  integrity="blake2b-256:..."></script>
```

or inline. The loader accepts kernel text (parsed at the level the page declares, K1 by default, K2 when graded clauses are used) or an encoded normal form. Because `k1nd1ng-norm` content-hashes the normal form, the integrity attribute is not an add-on: it *is* the program's identity, it keys the parse cache, and two pages that load the same process share one cached plan in `b3110ws`.

The surface language is compiled to the kernel ahead of time by the `f1r3lang` toolchain, as TypeScript is compiled to JavaScript. This keeps the compiler, the DDL and the cost-accounting transpiler out of the browser's trusted base in version 1 and makes `.knf` the web's unit of shipped behaviour. Compiling in the browser is Decision 1.

7.2 Imports are free names

A page's process is closed except for its free names, and CampF1R3's deploy path already grounds every free name to an unforgeable derived from its identifier. The browser changes only *what the grounding is derived from*: the capability broker supplies, for each free identifier the page uses, either a granted system name or a dead channel. The page's import list is therefore computable from its normal form before a single reduction, which lets the browser show the user what a page will ask for before it runs.

Free name	Granted capability
doc	the document root (Section 7.3)
net	fetch, restricted to the page's origin unless widened by grant
store	per-origin persistent space: a CampF1R3 checkpoint saved by the browser
clock	frame ticks and timers
log	the developer console
shard	the shard bridge, only after an explicit user grant

7.3 The document is a namespace

Every element the page can reach is represented by an unforgeable name minted by the bridge. The bridge keeps a table from those names to blitz-dom node ids; the table never leaves the bridge. A page receives names only by asking for them on a name it already holds.

Principle 4 (Attenuation is the tree). Queries on an element's name return only that element's descendants. Holding a subtree's name therefore grants exactly that subtree, and handing a component a container's name confines it to the container.

This is the object-capability discipline, and here it costs nothing: it is what unforgeability already means. A third-party widget embedded in a page gets the name of its slot and cannot read the rest of the document, which today requires iframes and cross-origin machinery.

The protocol on an element's name is a small set of messages, each carrying a return channel where a reply is expected: `query` and `query1` (selectors, answered with names), `get` and `set` for properties and attributes, `class`, `append`, `replace` and `remove` with HTML fragments or structured terms, and `listen` to subscribe a channel to an event type. Reads are answered from the committed document, so a page never observes a half-applied frame.

7.4 Events are data on channels

`listen` installs a forwarder: each occurrence of the event is produced as a datum on the page's channel. Handlers are ordinary receipts, persistent when they should fire repeatedly. A lamp that toggles on click, in surface syntax:

```
new found, clicks, on, off in {
  doc!("query1", "#lamp", *found) |
  for (lamp ← found) {
    lamp!("listen", "click", *clicks) |
    off!(Nil) |
    for (_ ← clicks & _ ← off) { lamp!("class", "lit") | on!(Nil) } |
    for (_ ← clicks & _ ← on) { lamp!("class", "") | off!(Nil) }
  }
}
```

There is no state variable and no callback: the state is which token exists, and the join is the state machine. Because exactly one token exists, the two handlers never race. Bubbling and default actions are implemented by the bridge, not the page: an event's datum carries a reply name, a handler that wants to stop propagation or prevent the default says so on it, and the bridge applies the default after a deadline if no handler has answered.

7.5 The frame is the unit of commit

f1r3x-host already separates *what* reduces from *when* a decided commit is released. The page process drives `Session::poll(now)` from the shell's frame callback; a round's commits carry DOM messages; the bridge applies them to `blitz-dom` as one batch; Stylo restyles, Taffy lays out, and the painter draws. The host's two pacing modes map directly: *free-running* for ordinary pages, with the bounded buffer providing back-pressure when a page produces faster than frames can show; *lockstep* for the debugger, where the user steps one round at a time and the document updates with each.

Requirement 3 (Frame atomicity). Mutations produced within one admitted round **MUST** be applied to the document together, and style and layout **MUST** be computed only between rounds.

7.6 Determinism, replay and the debugger

In the Deterministic profile, CampF1R3's results and event log are bit-identical across targets and thread counts. A page's run is then a function of its normal form and the sequence of events injected at its apertures (user input, network replies, timer ticks). The browser records that sequence with the log. Three features follow at no further semantic cost:

- **Bug reports are replays.** A report is the page hash, the grants and the log; the developer re-runs it in the Replay profile and sees the same run.
- **Time travel.** CampF1R3's soft checkpoints let the debugger step back as well as forward.
- **A live view of the space.** `k1ndl1ng-norm::json` and the host's scene frames already describe the store for a visualiser; the same vocabulary serves a devtools pane, and the Rhobots headset client can attach to a page as easily as to the Mac server.

7.7 Cost accounting as a page budget

CampF1R3 hooks accounting at the COMM through an `Observer` that may abort. The browser installs a *page meter*: an observer debiting a per-tab budget replenished per frame, with a larger allowance for the foreground tab. A page that exhausts its budget stops making progress without stopping the browser, and the user sees why. This replaces the “page is not responding” heuristic with an exact quantity, bounds the damage from runaway or hostile code (including in-page mining), and uses the same cost model the shard charges for, so a developer can predict a deploy's cost from a local run.

7.8 Graded where-clauses arbitrate the page's races

Interfaces are full of races: two handlers for one keystroke, a stale network reply against a fresh one, an animation against an interaction. In JavaScript these are settled by event-loop ordering that the programmer does not control. In f1r3lang they are settled by the resolver, and the graded where-clause is the syntax in which the page arranges it. CampF1R3 already isolates choice in the oracle ω ; the proposal adds a Graded profile that evaluates each candidate's clause in the page's declared semiring V and selects accordingly.

Semiring V	Resolver	Use in a page
Booleans	any enabled candidate	today's rholang guards
Viterbi ($\max, \times$)	argmax, attained exactly	priorities: the autocomplete handler outranks the fallback, deterministically
tropical ($\min, +$)	least cost, attained exactly	choose the cheapest source: cached, local, or network
$\mathbb{R}_{\geq 0}$	sample the normalised clause	randomised layouts, experiments, load spreading
$\mathbb{R}_{\geq 0}$ with learning	sample, then update in the fibre	interfaces that adapt to the user locally, without a server

Two results from the choice-formulae note are directly useful. First, weak fairness is the combinator $\psi \oplus \varepsilon$: a page, or the browser on the page's behalf, can guarantee that no persistently enabled handler starves. Second, strength enters a program only at *apertures* or through reflection, so a static pass over a page's normal form bounds the choice strength of everything in its interior; the browser can refuse, or run in a restricted profile, a page whose interior exceeds a declared ceiling. The quantum instance $V = \mathbb{C}$ is out of scope for the browser; the resolver seam is where it would go.

Requirement 4 (Graded resolution is opt-in). A page that does not declare a semiring **MUST** run in the Deterministic profile. Replay of a graded page **MUST** record the resolver's selections in the log, so that sampled runs are reproducible.

8 The shard as the server

8.1 What the Web authenticates, and what it does not

The Web authenticates *channels*. TLS proves that the bytes arrived from a party holding a certificate for a hostname; it says nothing about whether those bytes are the ones the author published. Every party that terminates TLS on the author's behalf — a CDN edge, a load balancer, a compromised origin, a hosting provider, a misissued certificate — can substitute a resource, and the client cannot tell. Subresource Integrity covers only subresources named by a document that was itself trusted because of its channel. The root document, and every dynamic response, is authenticated by location alone. Sessions compound the problem: a cookie is a bearer token attached ambiently to every request to its domain, so whoever copies it is the user, and any page that can cause a request to that domain speaks with the user's authority.

Principle 5 (Authenticate content, not channels). The browser establishes the integrity of a resource from the resource and the shard's agreed state, never from the connection it arrived on. Transport encryption remains, for confidentiality; it stops being load-bearing for integrity.

Under this principle, a man in the middle can still withhold, delay or observe traffic. It can no longer substitute. The rest of this section shows how each kind of resource reaches that standard.

8.2 Three kinds of resource

Kind	Identity	How the client authenticates it
Immutable content (code, documents, media)	its hash: the K1ndl1ng normal-form hash for processes, BLAKE2b-256 of the bytes for blobs	recompute the hash; any transport will do
Mutable binding (“the current version of this site”)	a versioned-registry name: publisher key, project, version range	the binding is shard state, established by consensus and checked against a finalized block
Dynamic response (the result of a computation)	a deploy, or a session message (§8.4)	re-execute locally in the Replay profile, or accept an attested result at a stated assurance level

Immutable content is self-certifying. A page is named by the hash of its normal form, and a blob by the hash of its bytes. The client recomputes the hash on receipt, so it can fetch from the shard, from a mirror, from a peer, or from an ordinary CDN without trusting any of them. This turns existing caching infrastructure into an untrusted transport, which is what it should have been: hash-addressed resources never go stale, so they can be cached forever at every layer. Large blobs SHOULD be kept out of RSpace; the shard stores their hashes in bindings, and the bytes live in any content-addressed store.

Mutable bindings are consensus state. `f1r3node-rust` already carries the mechanism. Its versioned registry (`casper/src/main/resources/VersionedRegistry.rho`) implements names of the form

$$\text{rho:serve:}\langle \text{service ver} \rangle : \langle \text{pub key} \rangle : \langle \text{project} \rangle : \langle \text{project ver} \rangle$$

with semver resolution, deprecation, and notify channels. Its mutators take the publisher’s key from the *authenticated* deployer identity, so “nothing an attacker can send on the wire lets them mutate someone else’s entries”, as the contract’s own commentary puts it. A website is then a `rho:serve project`: the publisher’s key is the site’s identity, a version is a release, deprecation is how a vulnerable release is withdrawn, and the notify channel is how clients learn of it. DNS and certificate authorities disappear from the integrity path.

The client must still know that the binding it was shown is the agreed one. Blocks carry a `post_state_hash` over `rspace++`’s history, which is a radix trie hashed with BLAKE2b-256, and the node’s HTTP API answers `is-finalized` and `last-finalized-block` queries. What is missing is an endpoint returning an *inclusion proof*: the trie path from a registry entry to a finalized block’s post-state hash. With it, the browser becomes a light client: it checks the block’s validator signatures against a known bonded set and the path against the hash, and trusts no node. Without it, the browser falls back to asking several independent nodes and requiring agreement. This is Obligation 6.

Freshness. A valid but old binding is the one substitution content addressing does not prevent: an attacker can replay a superseded, correctly signed state. The browser therefore records, per publisher, the highest finalized block at which it has seen a binding, rejects proofs from older blocks, and subscribes to the notify channel of every project it has open.

8.3 Dynamic responses and the assurance ladder

A dynamic response in this architecture is the result of running a process against shard state. Because the client holds the same executive, it can in principle recompute any result whose inputs it can obtain with proofs. That gives a ladder of assurance, and the browser shows the user which rung a response stands on.

1. **Node-attested.** An exploratory deploy (read-only; the node bounds its concurrency, phlogiston and time) answered and signed by one node. Trust: that node.
2. **Quorum-attested.** The same query answered identically by k of n independent nodes. Trust: fewer than k colluding.
3. **Proof-carrying.** The result is data at a name in a finalized post-state, delivered with an inclusion proof. Trust: the bonded validator set.
4. **Re-executed.** The client fetches the process by hash and the state it reads with proofs, and replays it locally in the Replay or Deterministic profile. Trust: the client's own executive.

Rung 4 is available only because the client and the shard share a semantics; it depends on CampF1R3 and rspace++ agreeing, which is Obligation 3.

8.4 Sessions without cookies

In flr3lang a session is not an identifier attached to requests. It is a name that two parties hold and nobody else does:

```
// client, in the page
new session, ack in {
  svc!("open", *session, *ack) |
  for (_ ← ack) { session!("inbox", *ack) | for (@items ← ack) { ... } }
}

// service, published under rho:serve:...
contract svc(@"open", session, ack) = {
  ack!(Nil) |
  for (@"inbox", ret ← session) { ... ret!(items) }
}
```

Every property cookies were invented to approximate, and every attack they enable, changes character:

No ambient authority, so no cross-site request forgery.

A request carries authority only because it is sent on the session name. Another page, even one open in the next tab, does not hold that name and cannot cause a send on it.

No bearer token to steal.

There is no string whose possession is identity; within a runtime the name is unforgeable, and across the network it is bound to a key that pages never see (below).

No script to exfiltrate it.

There is no JavaScript, and a third-party component holds only the names it was granted.

No tracking side-channel.

Nothing is attached to requests by default, so there is no third-party cookie to disable.

The session is visible and killable.

It is a continuation in a store. Ending it is consuming the name or revoking its key; the browser can list a tab's open sessions as it lists its grants.

Its protocol can be typed.

A session is a channel with a behaviour, so the linear fragment and the spatial-behavioural logics already in the programme apply: “this session may be used once”, “this session answers inbox before it answers send” become checkable properties rather than server conventions.

Realising unforgeability across the network. Unforgeable names are a property of one runtime. Between the browser and the shard the name must be realised cryptographically, and the node already has the primitive: deploys are signed, and a contract can read the authenticated deployer's key through `rho:deploy:data` and `rho:system:deployerId:ops`, exactly as the versioned registry does. The session is therefore opened with a fresh key pair generated by the browser's key-store. The service keys the session's channel by that public key, and accepts a message on it only from a deploy signed by the matching private key. Replay of a captured request is refused because `DeployData` carries a timestamp, `validAfterBlockNumber` and an optional `expiration_timestamp` under the signature. The page holds the session *name*; the browser holds the *key*. Closing the tab can destroy the key, which ends the session irrevocably.

8.5 Three server tiers

A deploy costs phlogiston and waits for consensus, which is right for a purchase or a vote and wrong for every keystroke of a chat. The design therefore has three tiers, and a session can span them.

Tier	Runs where	Latency	Integrity
Client	the tab's CampF1R3 executive	a frame	the user's own executive
Session host	a CampF1R3 executive beside a node, reached over WebSocket (<code>f1r3x-serve</code> , whose sessions already outlive sockets)	a round trip	attested by the host; auditable later by replay of a log whose hash is anchored on the shard
Shard	<code>f1r3node-rust</code> consensus	finalization	the bonded validator set

The session host runs in the `Deterministic` profile and periodically deploys the hash of its event log to the shard. Nobody need trust it in the moment to hold it to account afterwards: any party with the log replays it and compares hashes. Interactive traffic lives on the session host; commitments that matter move to the shard.

8.6 Confidentiality

Integrity is gained; confidentiality is not automatic. Shard state is replicated to every validator and readable by anyone who can query a node, so session content committed there in clear is public. Private sessions MUST encrypt payloads end to end under keys derived from the session keys, or

keep private content on the session host and commit only hashes. Transport encryption continues to protect traffic in flight. The choice between these is Decision 5.

8.7 What remains trusted

Honesty about the residue: the browser binary and its update channel; the initial bonded-validator set or a checkpoint of it (the light client's root of trust, taken once, like a CA bundle but for one shard); the user's keystore; the user interface that shows grants, assurance levels and deploy costs; and availability, since a network attacker can still withhold. Traffic analysis is untouched.

9 Security model

Integrity comes from content and consensus.

See Section 8: hashes for immutable resources, finalized registry state for bindings, and a stated assurance level for dynamic responses.

Capabilities replace the same-origin policy.

A page has no ambient authority: no global object, no cookie jar it can read, no network beyond what its net name permits. Cross-origin cooperation is name passing, deliberately.

Grants are visible before execution.

Imports are free names in a normal form, so the browser knows a page's full requested authority at load time.

Reflection is code mobility.

Receiving a quoted process and dropping it (*y) runs code that arrived as data — the rho analogue of eval. The choice-formulae note shows the same construct carries choice strength into an interior that was checked without it. The browser therefore meters dropped code under the receiving page's budget and, for graded pages, enforces the declared ceiling on drops (the stratified where of that note).

Unforgeability is authority, not ownership.

A granted name can be copied to another process the page controls; the review of the quantum graded-where paper made the same distinction. Revocation is therefore by forwarder: the broker grants a proxy name it can sever, never the underlying system name.

The legacy tab is a different trust domain.

It gets no names; it talks to f1r3lang pages only through one channel of ground data mediated by the broker.

Memory safety.

The page process contains Rust code only; CampF1R3 forbids unsafe outside its FFI, which the native browser does not use.

10 Compatibility with the JavaScript web

The browser serves four kinds of content, and says which kind a tab holds.

1. **Native f1r3lang pages**, rendered by Blitz and driven by the executive.

2. **Static HTML and CSS** without scripts, rendered natively by Blitz, which already handles many such sites. Documentation, articles and much of the long tail of the web fall here.
3. **JavaScript pages**, opened in the legacy process under Servo (or CEF), visibly marked, with no access to f1r3lang capabilities.
4. **Mixed pages** in the reach tier: f1r3lang pages served to stock browsers run under f1r3x-wasm with a small loader, so one page serves both audiences.

WebAssembly modules are a possible later addition as foreign processes: a module instantiated by the browser, exposed to the page as a name, called by communication. This keeps computation-heavy libraries available without reintroducing a JavaScript host. It is not in the plan below.

11 Integration with existing web infrastructure

The greenfield design is only useful if it meets the Web where the Web already is. The question is when to convert. A f1r3lang resource is content-addressed, capability-scoped and replayable; a Web resource is location-addressed, ambiently authorised and opaque. Conversion in either direction changes which of those properties hold, so the decision is about which properties someone downstream will actually use.

11.1 Four rules

Principle 6 (Import pins, export projects). Bringing a Web resource in fixes it: fetch, hash, and from then on refer to it by hash. Sending a f1r3lang resource out produces a *projection*: a Web rendering that keeps what the Web can carry and drops the rest.

Principle 7 (Convert data and bindings, never behaviour). Documents, media, datasets and name bindings convert in both directions. JavaScript applications do not become f1r3lang processes, and f1r3lang processes do not become JavaScript; behaviour crosses only by running in its own engine (the legacy tab one way, f1r3x-wasm the other).

Principle 8 (Every projection points home). An exported resource carries the address it was projected from, so a native client that meets the projection can upgrade to the original, the way a client follows Alt-Svc to a better transport.

Principle 9 (Convert where a guarantee stops being used). Pin a Web resource when f1r3lang code will depend on it, because replay needs fixed inputs. Project a f1r3lang resource when its consumer cannot check what the projection drops. Otherwise leave each where it lives and cross by reference.

11.2 f1r3lang to the Web: when to project

Projection makes sense when the consumer is not a native client and the loss is acceptable for that consumer.

- **Audiences without the browser.** The reach tier serves the page itself under f1r3x-wasm; semantics survive, capability isolation weakens to whatever the host browser gives.
- **Machines that speak only HTTP.** Crawlers, link previews, mail clients, archivers and assistive tools need HTML. A gateway runs the page bridge headless — the same Blitz document, driven by the same executive, with no window — and serves the resulting document. That is server-side rendering with no second implementation.

- **Interoperating services.** A REST or JSON facade over registry names and data-at-name reads, as the node's HTTP API already partly is.
- **Static documents and media,** for which projection is essentially lossless if integrity travels with them.

Integrity can travel further than one might expect, because the Web has grown the pieces: a hash-bearing URL and an ETag equal to the hash make caching exact; Content-Digest (RFC 9530) states the hash in the response; HTTP Message Signatures (RFC 9421) let the gateway, or the publisher's key, sign it; and a Link: <f1r3://...>; rel="alternate" header points home. A projection then fails only against a client that ignores these headers, which is every stock browser today, and none of the ones that matter for verification.

Do not project capability-bearing interactions. A session projected to the Web becomes a cookie at the gateway: the gateway holds the session key and the user's authority reduces to a bearer token again. Where this cannot be avoided, the gateway **MUST** be named as the principal holding the key, and the session **SHOULD** be scoped to the least authority the Web client needs.

11.3 The Web to f1r3lang: when to pin

Pinning makes sense when a f1r3lang process will rely on the resource, or when someone will later need to prove what it was.

- **Assets consumed by f1r3lang pages.** Images, fonts, stylesheets and HTML fragments are fetched once, hashed, and referenced by hash thereafter. A page that names its assets by hash is replayable and immune to asset substitution.
- **Versioned dependencies and datasets,** which are immutable by intent; pinning only makes the intent enforceable.
- **Evidence.** "This URL served these bytes at this time" is worth pinning with a shard attestation for citations, contracts, journalism and disputes. It also defeats link rot.
- **Names.** A domain owner publishes, under DNSSEC or at an HTTPS well-known location, the publisher key and project it corresponds to. The browser verifies that mapping once, records it, and thereafter resolves the domain through the registry, so TLS is used once to bootstrap a binding rather than on every visit to vouch for content.
- **Identity.** Passkeys (WebAuthn) are already key pairs held by the user's device. A passkey can sign a delegation authorising a shard session key, which moves a user's existing Web identity into the session model without a password or a cookie. WebAuthn typically signs with P-256 while deploys use secp256k1, so the bridge is a delegation, not a reuse of the key; this is Obligation 8.

Web services as apertures. A Web API that a f1r3lang page calls is wrapped as a process behind a name. Its answers cannot be made trustworthy by wrapping them, and the design should not pretend otherwise: they enter at an aperture, in the sense of the choice-formulae note, and are marked as unattested inputs in the page's log. Protocols that attest TLS sessions to third parties (TLSNotary- or DECO-style) could raise their assurance, and are worth evaluating later.

Do not pin personalised or rapidly changing responses (a feed, a search result) unless their value is as evidence, secrets of any kind, or JavaScript as behaviour.

11.4 Decision table

Resource	f1r3lang → Web (project)	Web → f1r3lang (pin)
Documents, media	Yes: hash URL, Content-Digest, signature, link home	Yes, when a page depends on them or as evidence
Site name	Publish a well-known mapping so Web users and crawlers find the site	Yes: bootstrap from DNS or HTTPS once, then resolve through the registry
Behaviour	Only by running it: f1r3x-wasm	Never translate; run in the legacy tab
Read APIs	JSON facade over registry and data-at-name	Wrap as an aperture process; mark results unattested
Sessions	Avoid; if forced, the gateway is the named principal	Bridge Web identity by passkey delegation to a session key
Dynamic content	Headless render for crawlers and previews	Do not pin, except as evidence

11.5 What the existing infrastructure becomes

The effect of these rules is that the Web’s infrastructure is reused but demoted. CDNs, proxies and caches keep carrying bytes, now as untrusted transport for hash-addressed content. DNS keeps naming things for humans, but only bootstraps bindings. TLS keeps encrypting traffic, but stops vouching for content. Certificate authorities stop being on the integrity path of a f1r3lang site at all. Web identity enters through passkeys; Web services enter as apertures. Nothing has to be torn down for the new model to be the stronger one.

12 Development plan

Five phases, each ending at a gate with a demonstrable criterion (Figure 2). Durations are planning estimates that assume three to four Rust engineers and should be re-estimated after Phase 0; the reach tier runs in parallel and is small.

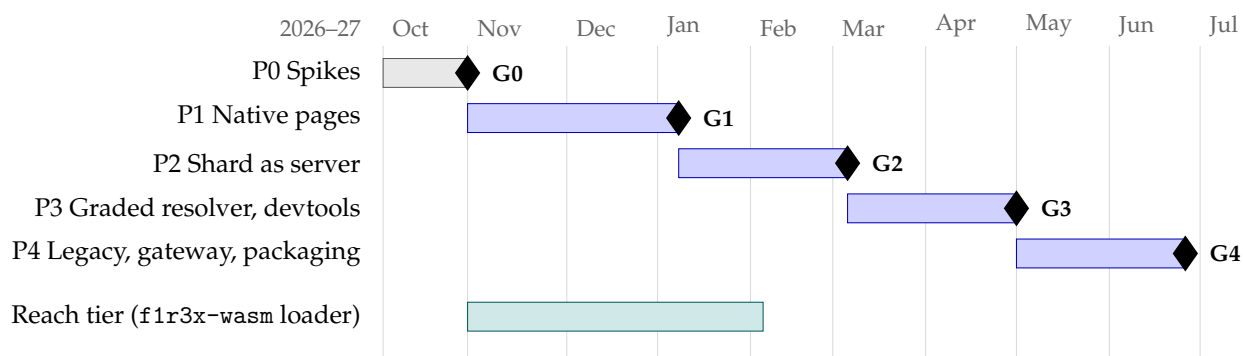


Figure 2: Indicative schedule from 5 October 2026, weeks to scale. Diamonds are gates; their criteria are in the text.

P0 Spikes (4 weeks).

Link blitz-dom and the CampF1R3 executive in one binary; mutate a document from a K1

program through a minimal bridge; measure the wasm size of the executive and the interpretation cost of a DOM-heavy workload in b3110ws. *Gate G0*: a K1 program changes rendered text in a Blitz window, and the measurements are written down.

P1 Native pages (10 weeks).

The page loader and the .knf format; free-name grounding through a stub broker; the DOM protocol; events as data with bubbling and default actions; frame-atomic commit; the Deterministic profile with event recording. *Gate G1*: TodoMVC written in f1r3lang, and a recorded session that replays identically.

P2 Capabilities and the shard as server (8 weeks).

The capability broker with user prompts and revocable forwarders; net and store; the key-store and signed deploys; publishing and resolving rho:serve sites; hash-verified fetch of pages and blobs; key-bound sessions; the node inclusion-proof endpoint and the light-client check (Obligation 6), with quorum reads as the interim; the page meter. *Gate G2*: a site published to a local three-validator shard loads through a deliberately hostile proxy that rewrites responses, and the browser refuses every rewritten resource; a session survives a replayed request without accepting it.

P3 Graded resolver and devtools (8 weeks).

K2 on in the executive; the Graded oracle profile for the Viterbi, tropical and $\mathbb{R}_{\geq 0}$ semirings; the fairness combinator; the ceiling check on drops; a devtools pane with replay, time travel and a live view of the space. *Gate G3*: a page using a sampled clause replays identically from its log.

P4 Legacy tab, gateway and packaging (8 weeks).

Servo in its own process behind one broker channel; the headless projection gateway with Content-Digest, signatures and links home; the session host with log anchoring; DNS and passkey bootstrap; the browser's own chrome rewritten as a f1r3lang page; installers for macOS, Linux and Windows. *Gate G4*: a public alpha.

Reach tier (parallel, about 14 weeks at low intensity).

A loader and a host-DOM implementation of the same protocol over f1r3x-wasm, so that the G1 TodoMVC runs unchanged in stock browsers.

13 Risks, obligations and open decisions

13.1 Risks

Risk	Mitigation
Blitz is beta with a small core team	Pin and vendor it; keep the bridge behind a trait over the mutation interface so a Servo-layout or in-house tree could substitute; contribute fixes upstream; the Stylo core is shared with Servo and Firefox
Interpreted normal forms are too slow for rich interfaces	The P0 measurement; the b3l10ws backend seam and the bytecode note's stated conditions for revisiting; batching DOM messages per round
Fine-grained DOM messages flood the store	Structured fragments in append and replace; coarse patch messages as an alternative (Decision 2)
Text input, IME, forms and accessibility	Inherited from Blitz and AccessKit; tested against the reach tier as an oracle
Graded clauses not yet in CampF1R3	K2 exists as a feature-gated level; P3 is scheduled after the native path works without it
Legacy-tab compatibility	CEF fallback behind the same broker channel

13.2 Obligations

Obligation 1. Measure, in P0, the frame-time cost of a representative interaction workload in b3l10ws, and state the threshold at which a compiled backend becomes necessary.

Obligation 2. Decide, from measurement, whether running the executive as wasm inside the page process is worth its cost as defence in depth.

Obligation 3. Establish, through the campf1r3-ospace adapter's differential tests, whether CampF1R3 replays rspace++ logs; client-side verification of shard results depends on it.

Obligation 4. Specify the Graded oracle profile in the CampF1R3 specification (as a profile beside Free, Deterministic and Replay), including how selections are logged for replay.

Obligation 5. Write the DOM protocol as a specification, with its reply conventions, error names and the attenuation rule, so the native bridge and the reach-tier bridge are tested against one document.

Obligation 6. Add to f1r3node-rust an endpoint returning an inclusion proof from a registry entry or data-at-name to a finalized block's post_state_hash, and specify the browser's light-client check (validator signatures against a bonded-set checkpoint, then the trie path).

Obligation 7. Fix one canonical hash for code stored in the registry: either the node records the K1ndl1ng normal-form hash beside each entry, or the client re-normalises the returned term and a differential test shows the two agree.

Obligation 8. Specify the passkey-to-session-key delegation, including the curve mismatch between WebAuthn's usual P-256 and the node's deploy signatures.

13.3 Open decisions

Open decision 1 (Compilation in the browser). Ship only kernel normal forms in version 1, as proposed, or include the surface f1r3lang compiler so that pages can ship source?

Open decision 2 (DOM message granularity). Fine-grained messages per element, as proposed, or whole-subtree patches carried as quoted processes, or both?

Open decision 3 (The document language). Keep HTML as the document language, or define it in the f1r3lang DDL so that the document is itself a term of a defined language, which the executive can already send and receive? The first is compatible; the second is the more natural endpoint.

Open decision 4 (Legacy engine). Servo, as proposed, or CEF from the start, trading a large C++ dependency for compatibility?

Open decision 5 (Private sessions). For confidential sessions, end-to-end encryption of payloads committed to the shard, or private content kept on the session host with only hashes committed?

Open decision 6 (Blob storage). Where hash-addressed bytes live: a blob endpoint on the node, a separate content store run by validators, third-party mirrors and CDNs, or all three?

Open decision 7 (Repository and name). A new repository under F1R3FLY-io, and a working name for the browser.

References

- [1] Dioxus Labs, *Blitz: a modular HTML/CSS rendering engine*, repository README (beta status, architecture, goals), accessed 28 September 2026. <https://github.com/DioxusLabs/blitz>
- [2] M. Larabel, “Servo browser engine making it easier for embedded use” (crate publication and LTS policy; 0.1 LTS), *Phoronix*, 13 April 2026. <https://www.phoronix.com/news/Servo-Embed-Crates-LTS>
- [3] M. Larabel, “Servo 0.5 released”, *Phoronix*, 2026. <https://www.phoronix.com/news/Servo-0.5-Released>
- [4] endoflife.date, *Electron* (release cadence and support policy). <https://endoflife.date/electron>
- [5] *Chromium Embedded Framework*, Wikipedia, and the cef Rust crate. https://en.wikipedia.org/wiki/Chromium_Embedded_Framework, <https://lib.rs/crates/cef>
- [6] J. Asmar, “From WKWebView to CEF: embedding Chromium in a Tauri app”, 22 May 2026. <https://getatrium.dev/blog/embedding-real-browser-auri>
- [7] *Tauri (software framework)*, Wikipedia. [https://en.wikipedia.org/wiki/Tauri_\(software_framework\)](https://en.wikipedia.org/wiki/Tauri_(software_framework))
- [8] Tauri pull request #15980, *tauri-runtime-cef*, September 2026. <https://github.com/tauri-apps/tauri/pull/15980>
- [9] OpenReplay, “Exploring Ladybird: the non-Chromium browser project” (April 2026 newsletter figures, alpha roadmap). <https://blog.openreplay.com/fr/ladybird-navigateur-non-chromium/>
- [10] L. G. Meredith, *CampF1R3*, *K1ndllng*, and *Bytecode, or the normal form?*, specifications in [publications/campf1r3](#), September 2026; implementation in [F1R3FLY-io/campf1r3](#).

- [11] L. G. Meredith, *Graded Where-Clauses*, and the choice-formulae note, publications/fuzzyware and publications/choice-types.
- [12] L. G. Meredith, *Graded Where-Clauses over the Reals*, publications/fuzzyware/graded-where-pln.tex.
- [13] L. G. Meredith, *Races Decided by the Born Rule*, publications/fuzzyware/graded-where-quantum-v6.tex.